R3.05 · TD-TP 09

PROGRAMMATION SYSTÈME

Les sockets TCP

BUT2 Informatique · R3.05 · 2026-2027 · Séance de 2 h

Au sous-sol du LAMAE tourne un vieux serveur que tout le monde appelle « l'Oracle ». Il ne parle qu'en TCP, ne répond qu'aux bonnes questions et garde quatre drapeaux. Pour l'interroger, il te faut d'abord construire ton propre client... puis ton propre serveur.

OBJECTIFS DE LA SÉANCE

- ☐ Comprendre le modèle client/serveur, l'adressage IPv4 (adresse + port) et l'ordre des octets du réseau.
- ☐ Écrire un **client TCP** : `socket`, `connect`, `write`/`read`, `close`.
- ☐ Écrire un **serveur TCP** itératif : `socket`, `bind`, `listen`, `accept`.
- ☐ Tester avec les outils du système : `nc` (netcat), `ss`, `strace`.
- ☐ Concevoir un petit **protocole applicatif** et traiter toute donnée reçue comme **hostile**.

FIL ROUGE Opération « Oracle »

Mission 1 : forger un client. Mission 2 : forger un serveur. Mission 3 : interroger l'Oracle (`code/sujet09/oracle_serveur.c`) avec ton propre client et capturer ses **quatre drapeaux** `FLAG{...}`. Mission 4 : auditer le « guichet d'accueil » d'un stagiaire, truffé de failles. Note tes drapeaux dans `badge09.txt`. Les exercices vont de ★ (incontournable) à ★★★ (pour aller loin).

Mission	Contenu	Durée
0 · Briefing	Modèle client/serveur, ports, ordre des octets	10 min
1 · Le client	Étapes 1 à 4 : `socket`, `connect`, test avec netcat, échange	35 min
2 · Le serveur	Étapes 5 à 7 : `bind`, `listen`, `accept`	30 min
3 · L'Oracle	Protocole texte, client ligne à ligne, 4 drapeaux	30 min
4 · L'audit	Chasse aux failles dans un serveur naïf	15 min
Bonus ★★★	Serveur multi-clients (`fork` ou threads), `getaddrinfo`, `tcpdump`	autonomie

RÈGLE D'OR DE LA SÉANCE

On compile avec les avertissements, et on ne teste **qu'en local** (`127.0.0.1`) : aucun programme de ce TP ne doit écouter sur le réseau de l'IUT.

compilation recommandée en R3.05

```
$ gcc -std=c17 -Wall -Wextra -pedantic -g prog.c -o prog
```

> **PIÈGE CLASSIQUE** **-std=c17 et les fonctions POSIX**
>
> Avec `-std=c17`, gcc n'expose que le C **standard** : les fonctions POSIX comme `inet_pton`, `inet_ntop`, `getaddrinfo` ou `sigaction` deviennent « implicitement déclarées » (avertissement, puis plantage possible). La règle du module : on écrit `#define _POSIX_C_SOURCE 200809L` **en toute première ligne** du fichier, avant tout `#include`. C'est ce que font tous les fichiers fournis dans `code/sujet09/`.

Mission 0 — Briefing : le modèle client/serveur

La notion de **socket** a été introduite dans les distributions UNIX de Berkeley (on parle de *sockets BSD*). C'est un mécanisme de communication entre processus (IPC) qui fonctionne aussi bien sur une même machine qu'à travers un réseau TCP/IP. Sous Windows, l'interface équivalente s'appelle *Winsock*.

Une socket est un **point de communication** par lequel un processus émet et reçoit des données. Pour les protocoles Internet, il est associé à une **adresse IP** et à un **numéro de port**. L'interface socket se place à l'entrée de la couche **Transport** (modèle DoD / TCP-IP) : on choisit donc entre deux protocoles.

Protocole	Caractéristiques
TCP (RFC 9293, ex-RFC 793)	Mode **connecté**, fiable (accusés de réception, retransmissions, ordre garanti), **flux d'octets** sans frontières de messages, bidirectionnel *full duplex*.
UDP (RFC 768)	Mode **non connecté**, sans garantie de livraison ni d'ordre, mais des **datagrammes** aux frontières préservées ; plus léger.

Ports et adresses

Un numéro de **port** (16 bits, donc 65 536 valeurs, le port 0 étant réservé) identifie le processus destinataire sur une machine, pour un protocole donné. Le **serveur** demande un port précis et connu à l'avance (exemple : 80 pour HTTP ; la liste usuelle est dans `/etc/services`) ; le **client** laisse en général le système lui attribuer un port libre quelconque, que le serveur découvre à la connexion.

Pour créer une socket, on précise trois caractéristiques :

1. le **domaine** de communication (famille de protocoles et format d'adresse) : `AF_INET` pour IPv4, `AF_INET6` pour IPv6, `AF_UNIX` pour une communication locale ;
2. le **type** : `SOCK_STREAM` (mode connecté, TCP), `SOCK_DGRAM` (non connecté, UDP), `SOCK_RAW` (accès direct à IP, ICMP...) ;
3. le **protocole** : `0` choisit le protocole par défaut du type (TCP pour `SOCK_STREAM`).

L'ordre des octets du réseau

Un entier de plusieurs octets peut être rangé « gros bout d'abord » (**big-endian**) ou « petit bout d'abord » (*little-endian*, cas des processeurs x86 et de la plupart des ARM). Les en-têtes réseau sont **toujours** en big-endian : c'est l'**ordre réseau**. On convertit avec `htons`/`htonl` (*host to network*, short 16 bits / long 32 bits) et dans l'autre sens avec `ntohs`/`ntohl`.

EX 0 — Repérage du terrain ★ 10 min

1. Affichez les pages de manuel du chapitre 7 : `man 7 socket`, `man 7 ip`, `man 7 tcp`, `man 7 udp`. Repérez dans `man 7 ip` la définition de `struct sockaddr_in`.
2. Listez les ports TCP en écoute sur votre machine : `ss -tln`. Cherchez dans `/etc/services` les ports de `ssh`, `http` et `https`.

3. Écrivez un petit programme qui affiche en hexadécimal `0x1234` puis `htons(0x1234)` (`#include <arpa/inet.h>`). Votre machine est-elle big-endian ou little-endian ?

Mission 1 — Forger un client TCP

Le scénario TCP est **dissymétrique** : le serveur se prépare et attend ; le client prend l'initiative de la connexion.

Client	Serveur
1. `socket()` : créer la socket	5. `socket()` puis `bind()` : s'attacher à un port connu
2. `connect()` : se connecter au serveur	6. `listen()` : se mettre en écoute passive
	7. `accept()` : récupérer une **socket de dialogue** par client
4. `write()`/`read()` : dialoguer	4. `read()`/`write()` : dialoguer
`close()`	`close()` de la socket de dialogue, puis retour à `accept()`

Étape 1 : créer la socket

Consultez `man 2 socket` : `int socket(int domain, int type, int protocol);` renvoie un **descripteur** (comme `open` ou `pipe`), ou -1 en cas d'erreur.

EX 1 **Première socket** ★ 5 min

Tapez **à la main** (pas de copier-coller : c'est en tapant qu'on retient) le programme `clientTCP-1.c`, compilez-le et exécutez-le. Quel numéro de descripteur obtenez-vous ? Pourquoi celui-là ? (Pensez à 0, 1 et 2.)

clientTCP-1.c

```c
#include <stdio.h>
#include <stdlib.h>        // exit, EXIT_FAILURE
#include <unistd.h>        // close
#include <sys/types.h>
#include <sys/socket.h>    // socket

int main(void)
{
    // AF_INET : IPv4 ; SOCK_STREAM : mode connecté ; 0 : protocole par défaut (TCP)
    int descripteurSocket = socket(AF_INET, SOCK_STREAM, 0);
    if (descripteurSocket == -1) {          // toujours tester le retour
        perror("socket");
        exit(EXIT_FAILURE);
    }
    printf("Socket créée avec succès ! (%d)\n", descripteurSocket);
    close(descripteurSocket);               // on rend la ressource
    return EXIT_SUCCESS;
}
```

On aurait pu préciser explicitement le protocole avec `IPPROTO_TCP` au lieu de `0`.

Étape 2 : se connecter au serveur

Consultez `man 2 connect` : `int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen);`. L'adresse du serveur dépend du domaine. L'interface propose une structure **générique** `struct sockaddr`, et le domaine IPv4 une structure compatible, `struct sockaddr_in` (`man 7 ip`) :

```c
struct sockaddr_in {
    sa_family_t    sin_family;  // AF_INET
    in_port_t      sin_port;    // numéro de port, en ORDRE RÉSEAU
    struct in_addr sin_addr;    // adresse IPv4, en ORDRE RÉSEAU
};
struct in_addr {
    uint32_t       s_addr;
};
```

On remplit une `sockaddr_in`, puis on passe son adresse à `connect` en la convertissant en `struct sockaddr *` (c'est du polymorphisme « à la C »). Pour la remplir :

- `htons(5000)` convertit le port dans l'ordre réseau ;
- `inet_pton(AF_INET, "127.0.0.1", &adr.sin_addr)` convertit l'adresse écrite en notation décimale pointée vers sa forme binaire ; elle renvoie **1** si tout va bien, **0** si la chaîne est invalide.

clientTCP-2.c (extrait de l'étape 2)

```c
struct sockaddr_in pointDeRencontreDistant;
socklen_t longueurAdresse = sizeof pointDeRencontreDistant;
memset(&pointDeRencontreDistant, 0, longueurAdresse);       // tout à zéro d'abord
pointDeRencontreDistant.sin_family = AF_INET;
pointDeRencontreDistant.sin_port = htons(5000);
if (inet_pton(AF_INET, "127.0.0.1", &pointDeRencontreDistant.sin_addr) != 1) {
    fprintf(stderr, "Adresse IPv4 invalide\n");
    close(descripteurSocket);
    exit(EXIT_FAILURE);
}
if (connect(descripteurSocket, (struct sockaddr *) &pointDeRencontreDistant,
            longueurAdresse) == -1) {
    perror("connect");
    close(descripteurSocket);
    exit(EXIT_FAILURE);
}
printf("Connexion au serveur réussie avec succès !\n");
```

EX 2 **Frapper à une porte fermée** ★ 5 min

Complétez `clientTCP-2.c` (la version complète, qui accepte l'adresse et le port en arguments, est dans `code/sujet09/`). Exécutez-le : vous obtenez `connect: Connection refused`. Pourquoi ? Que se passe-t-il si vous donnez l'adresse `1.2.3` ?

Étape 3 : vérifier la connexion avec netcat

Pour tester un client, il faut un serveur. L'outil **netcat** (`nc`) en fournit un en une ligne. Dans un **second terminal** :

TERMINAL

```
$ nc -l 5000                # écoute sur le port 5000 (variante : nc -l -p 5000)
```

TERMINAL

```
$ ./clientTCP-2
Socket créée avec succès ! (3)
Connexion au serveur réussie avec succès !
```

Dans l'architecture client/serveur, c'est le **client qui a l'initiative** : le serveur doit être en écoute **avant** que le client ne se connecte.

Étape 4 : échanger des données

TCP est **bidirectionnel** (*full duplex*) et **orienté flux d'octets**. On échange des octets avec `read`/`write` (comme sur un fichier ou un tube) ou avec `recv`/`send`, qui ajoutent un paramètre `flags` (`man 2 send`). Les octets échangés devraient respecter un **protocole applicatif** ; pour ce test, la règle est simple : le client envoie une chaîne, le serveur répond `ok`.

clientTCP-3.c (extrait de l'étape 4)

```c
#define LG_MESSAGE 256
char messageEnvoi[LG_MESSAGE];
char messageRecu[LG_MESSAGE];

snprintf(messageEnvoi, sizeof messageEnvoi, "Hello world !\n");
ssize_t ecrits = write(descripteurSocket, messageEnvoi, strlen(messageEnvoi));
if (ecrits == -1) {
    perror("write");
    close(descripteurSocket);
    exit(EXIT_FAILURE);
}
printf("Message %s envoyé avec succès (%zd octets)\n", messageEnvoi, ecrits);

// On lit AU PLUS LG_MESSAGE - 1 octets pour garder la place du '\0'
ssize_t lus = read(descripteurSocket, messageRecu, sizeof messageRecu - 1);
switch (lus) {
case -1:
    perror("read");
    close(descripteurSocket);
    exit(EXIT_FAILURE);
case 0:                         // 0 = fin de flux : le serveur a fermé
    fprintf(stderr, "La socket a été fermée par le serveur !\n");
    break;
default:
    messageRecu[lus] = '\0';  // read ne termine PAS la chaîne
    printf("Message reçu du serveur : %s (%zd octets)\n", messageRecu, lus);
}
```

EX 3 — Premier dialogue

★ 15 min

1. Lancez `nc -l 5000`, puis `./clientTCP-3`. Dans le terminal de netcat, le message `Hello world !` apparaît : tapez `ok` puis Entrée. Que reçoit le client, et combien d'octets ? Pourquoi 3 ?
2. Recommencez, mais cette fois arrêtez netcat avec **Ctrl-C** au lieu de répondre. Quelle branche du `switch` est exécutée ?
3. Observez les appels système avec `strace -e trace=network,read,write ./clientTCP-3`.
4. Lisez `man 2 shutdown` : `int shutdown(int sockfd, int how);` permet de fermer **un seul sens** de la communication (`SHUT_WR` : « j'ai fini d'écrire »). Dans quel cas est-ce plus utile que `close` ?

> **SÉCURITÉ** **read ne fabrique pas de chaîne de caractères**
>
> `read` copie des **octets**, sans ajouter de `'\0'`. Si l'on lit `sizeof messageRecu` octets puis qu'on affiche avec `%s`, `printf` lit **au-delà du tableau** jusqu'à trouver un zéro (fuite d'information, plantage). La version d'origine du TP lisait les 256 octets du tampon : un serveur bavard suffisait à provoquer le débordement. Règle : lire au plus `taille - 1` octets, puis écrire `buf[lus] = '\0'`.

PIÈGE CLASSIQUE **TCP est un flux, pas une suite de messages**

Un `write` de 14 octets côté client **n'est pas** forcément reçu par un seul `read` de 14 octets côté serveur : le réseau peut découper (ou regrouper) les données. Sur `127.0.0.1` avec de petits messages, tout semble marcher… jusqu'au jour où ce n'est plus le cas. Un vrai protocole définit donc **où s'arrête un message** : un délimiteur (`'\n'`) ou une longueur annoncée, et on **boucle** sur `read` jusqu'à l'avoir entier. C'est tout l'enjeu de la mission 3.

Mission 2 — Forger un serveur TCP

Étape 5 : s'attacher à un port avec bind

Un serveur doit être joignable à une adresse et un port **connus**. L'appel `bind` (`man 2 bind`) attache la socket à une adresse locale : `int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);`. Pour une adresse IPv4 sur 32 bits, on convertit avec `htonl`. La constante `INADDR_ANY` signifie « toutes les interfaces de la machine » ; `INADDR_LOOPBACK` (127.0.0.1) limite l'écoute à la machine elle-même.

serveurTCP-1.c (extrait de l'étape 5)

```c
#define PORT 5000
struct sockaddr_in pointDeRencontreLocal;
memset(&pointDeRencontreLocal, 0, sizeof pointDeRencontreLocal);
pointDeRencontreLocal.sin_family = AF_INET;
pointDeRencontreLocal.sin_addr.s_addr = htonl(INADDR_ANY);  // toutes les interfaces
pointDeRencontreLocal.sin_port = htons(PORT);
if (bind(socketEcoute, (struct sockaddr *) &pointDeRencontreLocal,
         sizeof pointDeRencontreLocal) == -1) {
    perror("bind");
    close(socketEcoute);
    exit(EXIT_FAILURE);
}
```

EX 4 **Un port, un seul processus** ★ 5 min

Compilez `serveurTCP-1.c` (il dort 2 secondes après le `bind`), puis lancez-le deux fois de suite : `./serveurTCP-1 & ./serveurTCP-1`. Expliquez le message `bind: Address already in use`. Un numéro de port identifie **un** processus communicant (pour un protocole donné).

SÉCURITÉ **Écouter sur INADDR_ANY, c'est ouvrir sa porte au réseau**

Un serveur attaché à `INADDR_ANY` est joignable par **toutes** les machines qui peuvent atteindre la vôtre. Pendant le développement, attachez vos serveurs à `127.0.0.1` (`htonl(INADDR_LOOPBACK)` ou `inet_pton`) : un serveur en cours d'écriture est, par définition, plein de bugs. Les ports inférieurs à 1024 sont **privilégiés** (réservés à root sous Linux) : utilisez des ports supérieurs à 1024.

Étape 6 : se mettre en écoute avec listen

`int listen(int sockfd, int backlog);` (`man 2 listen`) transforme la socket en **socket d'écoute passive**. Le paramètre `backlog` fixe la taille de la file des connexions **établies mais pas encore acceptées** par `accept`.

serveurTCP-2.c (extrait de l'étape 6)

```c
#define TAILLE_FILE 5
if (listen(socketEcoute, TAILLE_FILE) == -1) {
    perror("listen");
    close(socketEcoute);
    exit(EXIT_FAILURE);
}
printf("Socket placée en écoute passive ...\n");
```

> **CULTURE** **La file d'attente et le déni de service**
>
> Si la file est pleine, les nouvelles demandes de connexion sont ignorées ou refusées : le serveur ne peut plus servir de nouveaux clients, c'est un **déni de service** (*Denial of Service*). Une attaque classique, le *SYN flood*, consiste à remplir cette file de demandes jamais terminées ; Linux s'en protège avec les *SYN cookies* (`/proc/sys/net/ipv4/tcp_syncookies`).

Étape 7 : accepter les connexions avec accept

`int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);` (`man 2 accept`) attend un client et renvoie une **nouvelle socket, dite de dialogue**, connectée à ce client. La socket d'écoute reste inchangée et peut accepter d'autres connexions. Le paramètre `addrlen` est un paramètre **valeur-résultat** : on y met la taille de la structure, `accept` y écrit la taille réellement utilisée.

Ce serveur est **itératif** : il sert un client après l'autre. Servir plusieurs clients en même temps demande du parallélisme (`fork` ou threads : voir les bonus).

serveurTCP-3.c (extrait de l'étape 7)

```c
for (;;) {                          // en théorie, un serveur attend indéfiniment
    struct sockaddr_in pointDeRencontreDistant;
    socklen_t longueurAdresse = sizeof pointDeRencontreDistant;  // à CHAQUE tour
    int socketDialogue = accept(socketEcoute,
                                (struct sockaddr *) &pointDeRencontreDistant,
                                &longueurAdresse);               // bloquant
    if (socketDialogue == -1) {
        perror("accept");
        continue;                   // un client raté n'arrête pas le serveur
    }
    char ipClient[INET_ADDRSTRLEN];
    inet_ntop(AF_INET, &pointDeRencontreDistant.sin_addr, ipClient, sizeof ipClient);
    printf("Client connecté : %s:%u\n", ipClient,
           (unsigned) ntohs(pointDeRencontreDistant.sin_port));

    ssize_t lus = read(socketDialogue, messageRecu, sizeof messageRecu - 1);
    if (lus > 0) {
        messageRecu[lus] = '\0';
        printf("Message reçu : %s (%zd octets)\n", messageRecu, lus);
        snprintf(messageEnvoi, sizeof messageEnvoi, "ok\n");
        if (write(socketDialogue, messageEnvoi, strlen(messageEnvoi)) == -1)
            perror("write");
    } else if (lus == 0) {
        fprintf(stderr, "La socket a été fermée par le client !\n");
    } else {
        perror("read");
    }
    close(socketDialogue);          // on ferme le dialogue et on attend le suivant
}
```

EX 5 Le serveur répond ★ 15 min

1. Complétez `serveurTCP-3.c` (création, `bind`, `listen`, puis la boucle ci-dessus), lancez-le et exécutez **deux fois** `./clientTCP-3`. Testez aussi avec `nc 127.0.0.1 5000` et `telnet 127.0.0.1 5000`.
2. Pendant que le serveur tourne, tapez `ss -tlnp | grep 5000`, puis `ss -tnp` pendant une connexion. Repérez la socket d'écoute et la socket de dialogue.
3. Connectez un `nc 127.0.0.1 5000` **sans rien taper**, puis lancez `./clientTCP-3` dans un troisième terminal. Que se passe-t-il ? Pourquoi est-ce un problème pour un serveur itératif ?
4. Arrêtez le serveur avec Ctrl-C et relancez-le aussitôt : si `bind` échoue, lisez l'encadré culture sur `TIME_WAIT` en fin de sujet et expliquez le rôle de `setsockopt(..., SO_REUSEADDR, ...)`.

> **PIÈGE CLASSIQUE** **Les deux bugs du serveur d'origine**
>
> Dans la version d'origine de `serveurTCP-3.c`, le cas `read == 0` (« client parti ») faisait `return 0;` : le **serveur entier s'arrêtait** dès qu'un client se déconnectait sans parler. Et `longueurAdresse` n'était pas réinitialisée avant chaque `accept` : comme c'est un paramètre valeur-résultat, une valeur rétrécie par un appel précédent peut tronquer l'adresse suivante. Un serveur ne meurt pas à cause d'**un** client.

Mission 3 — Interroger l'Oracle

L'Oracle (`code/sujet09/oracle_serveur.c`) est un serveur TCP à **protocole texte** : une commande par ligne, terminée par `'\n'`, et **une ligne de réponse** par commande. Interdiction de lire son code source avant d'avoir capturé les quatre drapeaux !

Commande envoyée	Réponse de l'Oracle
(à la connexion)	`ORACLE LAMAE v1 - tapez AIDE`
`AIDE`	la liste des commandes
`ENIGME n` (n de 1 à 4)	`ENIGME n : <question>`
`REPONSE n mot`	`BRAVO FLAG{...}` ou `FAUX (k/10)`
`QUITTER`	`AU REVOIR`, puis fermeture
ligne de plus de 127 caractères	`ERREUR ...`, puis fermeture

```
• • •  TERMINAL
$ gcc -std=c17 -Wall -Wextra -pedantic -g oracle_serveur.c -o oracle_serveur
$ ./oracle_serveur              # écoute sur 127.0.0.1:5050
Oracle a l'ecoute sur 127.0.0.1:5050 (Ctrl-C pour arreter)
```

EX 6 Premier contact ★ 5 min

Connectez-vous à l'Oracle avec `nc 127.0.0.1 5050`, tapez `AIDE`, lisez l'énigme 1 et capturez le **premier drapeau**.

EX 7 Le client de l'Oracle ★★ 25 min

Écrivez `oracle_client.c` : il se connecte à l'Oracle (adresse et port en arguments, défauts `127.0.0.1` et `5050`), affiche la bannière, puis, en boucle, lit une commande au clavier avec `fgets`, l'envoie, lit **une ligne** de réponse et l'affiche, jusqu'à `AU REVOIR` ou la fermeture par le serveur. Écrivez d'abord ces deux fonctions :

```c
/* Envoie les lg octets de texte, en bouclant : send peut n'en envoyer qu'une partie.
 * Retourne 0 si tout est parti, -1 en cas d'erreur. */
int envoyerTout(int fd, const char *texte, size_t lg);

/* Lit une ligne sur fd, au plus taille-1 caractères, sans le '\n', terminée par '\0'.
 * Retourne la longueur lue, 0 si le serveur a fermé, -1 en cas d'erreur,
 * -2 si la ligne est trop longue pour le tampon. */
ssize_t lireLigne(int fd, char *buf, size_t taille);
```

Capturez les **quatre drapeaux** avec votre client et notez-les dans `badge09.txt`.

INDICE

La façon la plus simple (pas la plus rapide) d'écrire `lireLigne` : lire **un octet à la fois** avec `recv(fd, &c, 1, 0)` jusqu'au `'\n'`, en vérifiant à chaque octet qu'il reste de la place. Pour l'envoi, passez le flag `MSG_NOSIGNAL` à `send` : si le serveur a fermé, `send` renvoie -1 (`errno == EPIPE`) au lieu de tuer votre processus avec le signal `SIGPIPE`.

PIÈGE CLASSIQUE **fgets et les lignes trop longues**

Si l'utilisateur tape plus de caractères que le tampon de `fgets`, la ligne lue ne se termine pas par `'\n'` et la suite reste dans `stdin`. Vérifiez la présence du `'\n'` avant d'envoyer, sinon l'Oracle recevra une commande coupée en deux.

DÉFI **Le rétro-ingénieur ★★★**

Les drapeaux de l'Oracle sont chiffrés dans l'exécutable : `strings oracle_serveur | grep FLAG` ne trouve rien. Sans lire le code source, retrouvez un drapeau avec **gdb** : `gdb ./oracle_serveur`, `info functions`, posez un point d'arrêt sur la fonction qui semble déchiffrer, connectez-vous, répondez juste, puis remontez dans la fonction appelante avec `finish` et affichez avec `print` la variable qui a reçu le résultat. Moralité : un secret caché dans un binaire n'est **jamais** un secret.

Mission 4 — L'audit du guichet d'accueil

Un stagiaire a écrit `code/sujet09/serveur_naif.c`, le « guichet d'accueil » du LAMAE (port 5060, sur 127.0.0.1 uniquement). Protocole : le client envoie `TAILLE n` suivi d'un retour à la ligne, puis exactement `n` octets (son pseudo) ; le serveur répond `Bienvenue <pseudo> (visiteur numero k)`.

```
$ gcc -std=c17 -Wall -Wextra -pedantic -g -fsanitize=address serveur_naif.c -o serveur_naif
$ ./serveur_naif &
$ printf 'TAILLE 4\nLuke' | nc -q1 127.0.0.1 5060
$ python3 -c "import sys; sys.stdout.write('TAILLE 60\n' + 'B' * 60)" | nc -q1 127.0.0.1 5060
```

EX 8 — Chasse aux failles ★★ 15 min

1. Le premier test devrait afficher `Bienvenue Luke`. Qu'obtenez-vous ? Expliquez à l'aide du piège « TCP est un flux » de la mission 1.
2. Lancez le second test : lisez le rapport d'AddressSanitizer. Quelle ligne déborde, et pourquoi ?

3. Le code compile **sans aucun avertissement**. Trouvez au moins **cinq** problèmes, repérés `(a)` à `(d)` dans le code, plus un qui ne se voit pas dans `accueillir`. Pour chacun, dites ce qu'un client malveillant peut en faire.
4. Corrigez le serveur (`serveur_corrige.c`) et rejouez les tests, plus `printf 'TAILLE -5\nxx'` et un client qui se connecte sans rien envoyer.

SÉCURITÉ **Les règles d'or du code réseau**

- **Toute donnée reçue est hostile** : longueur, valeurs numériques, caractères. On valide **avant** d'utiliser (bornes, signe, format, `'\0'` final).
- Une longueur annoncée par le client ne sert **jamais** directement de taille de copie dans un tampon.
- On boucle sur `read`/`recv` et `write`/`send` : les transferts partiels sont normaux.
- On teste **tous** les retours (`socket`, `bind`, `listen`, `accept`, `read`, `write`, `inet_pton`...).
- Copies et formatages **bornés** : `snprintf`, jamais `sprintf`/`strcpy` vers un tampon fixe ; jamais une donnée reçue comme format de `printf`.
- Un délai d'inactivité (`SO_RCVTIMEO`, `man 7 socket`) empêche un client muet de bloquer le service.
- On n'exécute jamais une commande construite à partir de données reçues (`system`, `popen`).

Ces règles reprennent celles du guide ANSSI sur les entrées non fiables et les copies de tampons.

POUR LES RAPIDES **Le serveur aux mille clients** ★★★

1. **Avec fork** (lien sujet 06) : après `accept`, le serveur crée un fils qui ferme la socket d'écoute, sert le client puis se termine avec `_exit` ; le père ferme la socket de dialogue et retourne à `accept`. Installez un gestionnaire de `SIGCHLD` avec `sigaction` (et `SA_RESTART`) qui appelle `waitpid(-1, NULL, WNOHANG)` en boucle : sinon, chaque client terminé laisse un **zombie**. Limitez le nombre de fils simultanés : sans limite, un attaquant qui ouvre des milliers de connexions épuise les processus.
2. **Avec des threads** (lien sujet 08) : un thread détaché (`pthread_detach`) par client, avec une structure allouée par client contenant son descripteur. Piège : passer `&fd` (variable de la boucle) au thread.
3. **Client HTTP avec getaddrinfo** : `getaddrinfo` (`man 3 getaddrinfo`) traduit un nom (`localhost`) en adresses IPv4 **ou** IPv6. Écrivez un client qui envoie `GET / HTTP/1.1`, `Host: localhost`, `Connection: close`, puis lit jusqu'à la fermeture. Testez-le contre un serveur web **local** : `python3 -m http.server 8000 --bind 127.0.0.1`.
4. **Voir les paquets** : `sudo tcpdump -i lo -n port 5050` pendant un dialogue avec l'Oracle. Repérez la poignée de main en trois temps (`[S]`, `[S.]`, `[.]`) et la fermeture (`[F.]`). Vos données circulent **en clair**.

Bilan de la séance

À RETENIR

- Client : `socket` → `connect` → `read`/`write` → `close`. Serveur : `socket` → `bind` → `listen` → `accept` (une socket de dialogue par client) → `close`.
- Adresses et ports sont stockés en **ordre réseau** : `htons`, `htonl`, `inet_pton` ; dans l'autre sens `ntohs`, `inet_ntop`.
- TCP est un **flux** : un protocole applicatif doit délimiter les messages, et on boucle sur les lectures et écritures.
- `read` ne termine pas les chaînes ; une donnée reçue n'est jamais fiable.
- On teste en local (`127.0.0.1`), avec `nc`, `ss`, `strace`.
- `#define _POSIX_C_SOURCE 200809L` en première ligne dès qu'on utilise des fonctions POSIX en `-std=c17`.

Le quiz du LAMAE (questions de révision)

- ☐ 1. Qu'est-ce qu'une socket ?
- ☐ 2. Quelles sont les trois caractéristiques d'une socket ?
- ☐ 3. Quelles sont les deux informations qui définissent un point de communication en IPv4 ?
- ☐ 4. Comment le serveur connaît-il le port utilisé par le client ?
- ☐ 5. Comment le client connaît-il le port utilisé par le serveur ?
- ☐ 6. À quelle couche du modèle DoD est reliée l'interface de programmation socket ?
- ☐ 7. Quel protocole de niveau Transport permet d'établir une communication en mode connecté ?
- ☐ 8. Quel protocole de niveau Transport permet d'établir une communication en mode non connecté ?
- ☐ 9. Quel est l'ordre des octets en réseau ?
- ☐ 10. À quels protocoles correspond le domaine `AF_INET` ? Est-ce le seul utilisable avec l'interface socket ? Citez-en au moins un autre.
- ☐ J'ai mes 4 drapeaux de l'Oracle dans `badge09.txt`.
- ☐ Mes programmes compilent sans **aucun** avertissement et ne testent qu'en `127.0.0.1`.

Aller plus loin

Ressources

Ressource	Pour quoi faire
`man 2 socket`, `bind`, `listen`, `accept`, `connect`	Les appels système du TP, avec toutes leurs erreurs possibles (`ECONNREFUSED`, `EADDRINUSE`...).
`man 2 send`, `recv`, `shutdown`, `setsockopt`	Les drapeaux (`MSG_NOSIGNAL`, `MSG_PEEK`...), la fermeture d'un seul sens, les options de socket.
`man 3 getaddrinfo`, `inet_pton`, `inet_ntop`, `byteorder`	Résolution de noms, conversions d'adresses et d'ordre des octets.
`man 7 socket`, `ip`, `ipv6`, `tcp`, `udp`, `unix`	Le fonctionnement des familles et des protocoles sous Linux.
M. Kerrisk, *The Linux Programming Interface*, ch. 56 à 61	La référence : sockets UNIX et Internet, serveurs, options avancées.
W. R. Stevens, *UNIX Network Programming*, vol. 1	Le classique de la programmation réseau.
A. Tanenbaum, *Réseaux*	TCP, UDP, IP et l'architecture en couches.
J.-M. Rifflet, *La communication sous UNIX*	Référence francophone sur les IPC et les sockets.
Guide ANSSI, règles de programmation sécurisée en C	Validation des entrées, copies bornées, valeurs de retour.
`nc`, `ss -tlnp`, `strace -f -e trace=network`, `tcpdump -i lo`, Wireshark	Tester, observer les sockets, tracer les appels, voir les paquets.
Beej's Guide to Network Programming	Guide en ligne gratuit, clair, avec `getaddrinfo` et IPv6.

Pour approfondir

CULTURE **TIME_WAIT et SO_REUSEADDR**

Quand une connexion TCP se ferme, le côté qui ferme en premier garde la connexion dans l'état `TIME_WAIT` pendant environ une minute (deux fois la durée de vie maximale d'un segment), pour que d'éventuels paquets retardataires ne soient pas pris pour ceux d'une nouvelle connexion. Un serveur relancé aussitôt peut alors échouer avec `EADDRINUSE`. L'option `SO_REUSEADDR`, posée **avant** `bind`, autorise la réutilisation du port : tous les serveurs réels l'utilisent.

CULTURE **Servir des milliers de clients**

Un processus ou un thread par client devient coûteux au-delà de quelques milliers de connexions (le célèbre problème « C10k »). Les serveurs modernes (nginx, Node.js...) utilisent le **multiplexage** : un seul thread surveille des milliers de sockets avec `poll` ou `epoll` (`man 7 epoll`) et ne traite que celles qui ont des données prêtes.

CULTURE **UDP, IPv6 et TLS**

En UDP (`SOCK_DGRAM`), pas de `listen` ni d'`accept` : on envoie et reçoit des datagrammes avec `sendto`/`recvfrom`, en précisant l'adresse à chaque fois. Pour être indépendant d'IPv4, on remplace `sockaddr_in` par `getaddrinfo` et `struct sockaddr_storage`. Enfin, TCP transporte les données **en clair** (vérifiez-le avec `tcpdump`) : tout protocole qui échange des mots de passe doit être chiffré avec **TLS** (bibliothèques OpenSSL, GnuTLS).

CULTURE **Un peu d'histoire**

L'interface socket est apparue en 1983 dans 4.2BSD, à l'université de Berkeley, financée par la DARPA pour diffuser TCP/IP. Elle a été reprise presque à l'identique par tous les systèmes, de Linux à Windows (Winsock), et normalisée par POSIX : le code de ce TP est vieux de plus de quarante ans dans ses principes.